

Technical Design Document (TDD)

F1tenth PoliMI

The simulator: gym, bridge and car models

Document type:	Technical Design Document (TDD)
Version:	0.02
Date:	November 9, 2026
Author:	F1tenth PoliMI team
Classification:	Public

Document History

Version	Date	Author	Changes
0.01	2026-10-25	F1tenth PoliMI team	Initial draft from `simulator.md` and `polimi_sim`
0.02	2026-11-08	F1tenth PoliMI team	Whole simulator design: the Jazzy and JAX gym target, the room world with a taped track, camera, IMU, evaluation monitors, the `/sim` namespace and truth frame, the config overlay; this document is the simulator's design home; steering limits

Document Approvals

Role	Name	Date	Signature
Technical Lead			
Project Manager			
Management			

Contents

1 Introduction . . . **3**
- 1.1 Purpose . . . 3
- 1.2 Scope . . . 3
- 1.3 References . . . 3
- 1.4 Status Conventions . . . 3

2 Architecture . . . **3**
- 2.1 Component View . . . 4
- 2.2 Components . . . 4
- 2.3 Deployment View . . . 5
- 2.4 Design Decisions . . . 5

3 Data . . . **7**
- 3.1 Configuration files . . . 7
- 3.2 Worlds . . . 8
- 3.3 Model parameters . . . 9
  - 3.3.1 `lidar_model` (`config/lidar/sl450.yaml`) . . . 9
  - 3.3.2 `actuation_model` (`actuation_model.yaml`) . . . 10
  - 3.3.3 `odom_model` (`odom_model.yaml`) . . . 10
  - 3.3.4 `teleop_bridge` (`teleop_bridge.yaml`) . . . 11
  - 3.3.5 `camera_model` . . . 11
  - 3.3.6 `imu_model` . . . 12
  - 3.3.7 Evaluation . . . 13
- 3.4 Ideal mode . . . 13

4 Interfaces . . . **14**
- 4.1 Simulator versus car . . . 14
- 4.2 Topics . . . 14

5 Requirements Traceability . . . **15**

6 Testing . . . **16**

7 Known Limits . . . **16**

8 Open Items . . . **17**

1 Introduction

1.1 Purpose

This document defines how the simulator stands in for the car: what the gym computes, which model nodes sit between the gym and the stack, how the world and the configs are described, how the simulator is scored, and where it still differs from the car. It is the simulator's design home: the fork's `polimi_sim/README.md` only says what the package is, how to run it on its own and how the config overlay works, and points here.

1.2 Scope

- **In scope:** f1tenth_gym and f1tenth_gym_ros as used here, our `polimi_sim` package (model nodes, world generator, evaluation, launch, configs), the simulator side of the car interface (`hw:=sim`).
- **Out of scope:** the stack and the car, in `TDD_F1tenth`; any perception or estimation (line detection, floor projection, SLAM, localization). How to run the simulator day to day is the runbook `docs/software/simulator.md`.

1.3 References

- `RASD_F1tenth`: FR-SIM-001 to 003, NFR-POR-001, AC-V1-02, AC-V1-04, AC-V2-B.
- `TDD_F1tenth`: the car interface (its Architecture and Interfaces sections), DEC-03 to DEC-10, the Jazzy migration.
- Runbook `docs/software/simulator.md`; host shortcuts in `scripts/sim/`.

1.4 Status Conventions

[Decided] agreed **[Planned]** decided, not built yet **[Deferred]** designed, built only when its trigger comes (Open Items) **[Proposed]** proposed, waiting for the team's confirmation **[TBD]** open [MEASURE] replace with a measurement on the car [VERIFY] check against the real driver or firmware.

> **Today versus target.** Built today: ROS 2 Humble, f1tenth_gym and f1tenth_gym_ros `dev-humble` pinned by `GYM_REF` and `GYM_ROS_REF` in `docker/ros/Dockerfile`, `polimi_sim` in this repo with the LiDAR, actuation, odometry and teleop models, walled tracks, and `ackermann_mux` inside `sim_car.launch.py`. Planned, in this order: the camera's replay sources (video, stream, bag), which need no gym; the Jazzy migration (`TDD_F1tenth`, Jazzy migration: the JAX gym `dev-jax`, the bridge `dev-jazzy`, `polimi_sim` in our fork `f1tenth_gym_polimi`), then the room world with a taped track, the ~~camera, the IMU and the evaluation monitors~~drawn camera and the track monitor. Planned parts are marked **[Planned]**. The IMU, the localization monitor and some camera and world extras are deferred until a decision needs them (S-12 to S-15).

2 Architecture

> **Key point.** The gym alone is too kind: a perfect scan 250 times a second, commands that act at once, a pose that is always exact. Code tuned on that breaks on the car. `polimi_sim` puts model nodes between the gym and the stack, so the stack runs against the car's topics, frames, rates and imperfections. The gym stays the source of physics and ground truth. The simulator produces sensor data and truth only: it contains no perception.

2.1 Component View

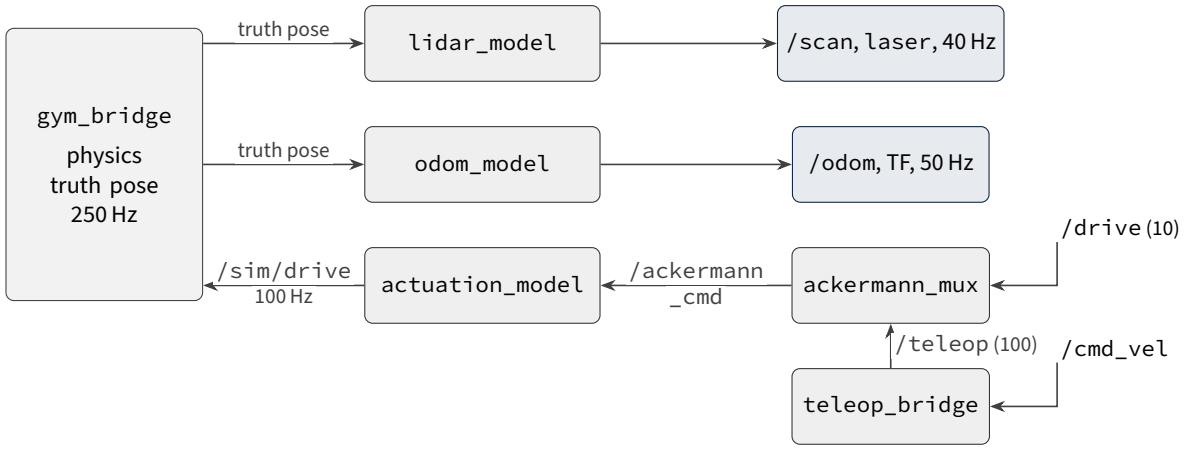


Figure 1: The simulator graph today. The gym computes where the car really is; three model nodes turn that into what the car's sensors would report and what its actuators would do. The mux is the car's own; it stays in the simulator through the Jazzy migration and moves to the stack's common layer with `TDD_F1tenth` DEC-08. The planned nodes are in table 1.

Read bottom-up: the stack publishes `/drive`, the mux lets teleop override it, `actuation_model` delays and limits the command as the VESC and servo would, and the gym moves the car. From the new true pose, `lidar_model` produces `/scan` and `odom_model` produces `/odom`. A static TF `base_link`→`laser` comes from `config/car/laser_mount.yaml`. The truth, `map`→`ego_racecar/base_link` and `/ego_racecar/odom` today (`sim_world` once planned, DEC-10), is for evaluation only.

2.2 Components

Component	Responsibility	Interfaces
f1tenth_gym	Physics: single-track model, collisions, ray caster	Python API
`gym_bridge` (f1tenth_gym_ros)	Runs the gym in ROS; truth pose and map	`/sim/drive` → truth TF, `/ego_racecar/odom`, `/map`, `/sim/scan_unused`
`lidar_model`	The SL450 as a rotating sensor with its errors	truth pose → `/scan`
`odom_model`	The car's `vesc_to_odom`, drift included	truth pose, `/sim/steering_command` → `/odom`, TF `odom`→`base_link`
`actuation_model`	The VESC and servo: delays, rates, limits, timeout	`/ackermann_cmd` → `/sim/drive`, `/sim/steering_command`
`teleop_bridge`	Keyboard and Foxglove teleop into the mux	`/cmd_vel` → `/teleop`
`ackermann_mux`	The car's mux and `mux.yaml`, until `TDD_F1tenth` DEC-08	`/teleop`, `/drive` → `/ackermann_cmd`

Component	Responsibility	Interfaces
`sim_car.launch.py`	Merges the configs, starts all of the above; `ideal:=true`, `camera_source:=` **[Planned]**	launch arguments

Planned component	Responsibility	Interfaces
`make_world`	World source file → every generated file (section 3.2)	CLI; run by the launch when the world is stale
`camera_model`	The Gemini 335L colour stream, drawn from the world or replayed from a video file or stream (section 3.3.5)	truth pose or `camera_uri` → `/camera/color/image_raw`, `camera_info`, preview
`world_publisher`	The tape as markers, for Foxglove	`/sim/truth/tape`
`track_monitor`	On-track status, laps, lateral offset, simulator health (section 3.3.7)	truth pose → `/sim/eval/*`, `/sim/health/*`, lap CSV
`sim_reset`	One way to put the car somewhere	`/sim/reset`, `/sim/reset_pose` → `/sim/initialpose`, `/odom_model/reset`
truth-jump detector	Tells resets (teleports) from wall contacts, so no model interpolates across them	truth pose, `/ego_racecar/collision`

Table 1: Planned components.

2.3 Deployment View

The simulator runs in the `arm` (Apple Silicon) and `x86` images only, never on the car (`jet`). Today the gym and the bridge are cloned at fixed commits during the image build, and `polimi_sim` and the configs are bind-mounted like the rest of the workspace.
After the Jazzy migration **[Planned]**: the fork `f1tenth_gym_polimi` (branch `dev-jazzy-polimi`) is the submodule `ros2_ws/src/f1tenth_gym_ros` and is built with `colcon` in the workspace; `polimi_sim` is a second package inside it, so the build and rosdep name its path explicitly. The JAX gym is pip-installed in the image and runs on the CPU (`JAX_PLATFORMS=cpu`). The `jet` image builds neither. The fork needs `f1tenth_system` next to it for `ackermann_mux` until DEC-09.
Foxglove connects on `ws://localhost:8765`; RViz through the optional noVNC container. Give the container at least 6 CPUs in Docker Desktop once the camera runs.

2.4 Design Decisions

IDs here are local to this document; `TDD_F1tenth`'s are written with its name.

ID	Decision	Rationale / alternatives
DEC-01	Model nodes between gym and stack; the gym stays physics and truth **[Decided]**	The stack meets the car's rates, frames and errors without changing the gym. Each model can be switched off (ideal mode) to see what it changes.
DEC-02	Each thing is defined once, in its own file **[Decided]**	Map, laser, mount and gym settings are chosen independently and merged at launch. The gym's own LiDAR takes its beams, angles, ranges and mount from the same laser and mount files, so the laser is never described twice.
DEC-03	The gym's LiDAR stays on, noise-free, on `/sim/scan_unused` **[Decided]**	Today its wall collision check runs on that scan; the JAX gym checks contact against wall segments instead. The stack's `/scan` comes only from `lidar_model`.
DEC-04	Two frame trees: truth apart from the estimate **[Decided]**	The truth tree is where the car is; `odom`→`base_link`→`laser` is where it thinks it is. Nothing joins them except SLAM, localization or the debug option. Today the truth hangs from `map`, which SLAM and AMCL also use; DEC-10 separates them.
DEC-05	`lidar_model` ray-casts with the gym's own caster **[Decided]**	From the truth pose composed with the mount, in blocks along the sweep, so motion distortion comes from real poses; the map matches the gym's. On the JAX gym, one exact ray–segment scanner (`SegmentScanSimulator2D`) per block, built at start **[Planned]**.
DEC-06	Ideal mode is the same graph with overrides **[Decided]**	Same nodes, topics and frames; only the imperfections go. The overrides live in `sim_car.launch.py` (`IDEAL_*`) and win over the parameter files.
DEC-07	Controllers read beam angles from each `LaserScan` **[Decided]**	`angle_min + i * angle_increment`, never hard-coded indices, so the same code runs with any laser and on the car.
DEC-08	`sim/gym.yaml` is the bridge's `config/sim.yaml` at the pinned commit, minus our keys **[Decided]**	Re-copied when the bridge is bumped; its header lists what differs. After the migration it is one of the fork's default configs (DEC-15).
DEC-09	The mux leaves the simulator after the Jazzy migration **[Proposed]**	Follows `TDD_F1tenth` DEC-08: the fork then runs from a bare clone, the stack's common layer owns the mux, and `teleop_bridge` publishes on `teleop_topic`. Through the migration the mux stays, so the simulator always has one.
DEC-10	The simulator's own names live under `/sim` and `sim_world` **[Planned]**	The truth frame becomes `sim_world` (a fork parameter `truth_frame`, default `map` as upstream); the gym's map is remapped to `/sim/map` and its reset input to `/sim/initialpose`. `/map`, `map` and `/initialpose` then belong to SLAM and AMCL, as on the car: giving AMCL an initial pose no longer teleports the car, and the stack cannot reach the truth through TF. The simulator starts Foxglove itself, outside the remaps (fork launch argument `start_foxglove`).

ID	Decision	Rationale / alternatives
DEC-11	One world source file generates everything the simulator needs **[Planned]**	Room, objects with a height, tape and start in one hand-edited file; a generator writes the collision map, the LiDAR slice, the camera scene and the tape truth from it, so a layout change is one edit and one command, and the LiDAR, the collisions and the camera never disagree.
DEC-12	The camera draws flat-shaded polygons with OpenCV, with no imperfections **[Planned]**	What closed-loop work needs is correct geometry (the real intrinsics and mount) and the driver's topics and timing. A per-pixel ray tracer, off-screen OpenGL, Gazebo and photoreal simulators were considered: slower, new dependencies or no Apple Silicon. Robustness to real images is developed on real video; imperfections are added only when a real-versus-simulated comparison shows the stack needs one (S-10).
DEC-13	The IMU is planar truth motion plus lever arm, noise and bias **[Deferred]**	Only the `odom` EKF (RASD OI-08), option B and exercise 5 use it, so it is built when one of them is decided (S-12). Body pitch and roll are not simulated.
DEC-14	Leaving the track is measured, not felt **[Planned]**	Tape stops nothing (RASD C-005), so `track_monitor` scores it from the truth: off track when the CoG's lateral offset exceeds $width/2 + tape\ width + track\ width/2$, i.e. all four wheels past a tape's outer edge for a car aligned with the track. Heading is ignored: a simpler stand-in for per-wheel contact patches.
DEC-15	Default configs in the fork, measured car configs in our repo **[Planned]**	The fork ships labelled defaults (stock f1tenth vehicle, datasheet sensors), never our measured car, so it runs on its own. Our `ros2_ws/config/` is passed as `config_dir`; for each file the launch takes ours if present, else the default, and logs which.
DEC-16	The simulator's design lives in this document **[Decided]**	One source of truth with release history; the fork's README stays short and points here (8 Nov 2026).

3 Data

3.1 Configuration files

All under `ros2_ws/config/`, bind-mounted: no rebuild. At launch the map, laser, mount and gym files are merged into the config the gym gets, written to `/tmp/polimi_sim/gym_<map>_<lidar>.yaml`.

File	Holds	Chosen with
`maps/<map>.yaml`	`map_path`, start pose `sx`, `sy`, `stheta`; any other `gym_bridge` parameter overrides `sim/gym.yaml`. For a generated world also a `world:` block (paths of the generated files) and the source hash **[Planned]**	`simcar <map>`
`lidar/<lidar>.yaml`	The laser: rate, FOV, resolution, ranges, effects	`lidar:=<lidar>` (default `sl450`)
`car/laser_mount.yaml`	`base_link`→`laser`: x 0.27, y 0, z 0.11 m, yaw 0, roll 0 [MEASURE]; read by the car too (`TDD_F1tenth` DEC-09)	–
`car/body.yaml`	**[Planned]** What the gym's vehicle parameters lack: steering limits left 0.3135 and right 0.2634 rad, track width 0.281 m [MEASURE]	–
`car/camera_mount.yaml`	**[Planned]** `base_link`→`camera_link` (x, y, z, roll, pitch, yaw); read by the car too	`camera_mount:=`
`camera/gemini_335l.yaml`	**[Planned]** Profile `720p30` (`720p60` after S-06), intrinsics and `plumb_bob` distortion [VERIFY]	`camera_profile:=`
`camera/<camera>.yaml`	**[Planned]** Calibration of a camera whose video is replayed (a phone, a webcam), as written by `camera_calibration`	`camera_info_url:=`
`sim/gym.yaml`	Everything else: topics, vehicle preset, opponents; `truth_frame: sim_world` **[Planned]**	–

Maps today: `spielberg` (default; Red Bull Ring, a closed track about 2.2 m wide) and `levine` (a corridor loop); the bridge also bundles `levine_obs`, `levine_blocked` and `blank`. They keep working after the room worlds arrive; the drawn camera, the IMU and the monitors start only on a map with a `world:` block. A replayed camera (`camera_source:=video` or `bag`) needs no world and no map.
Relative map paths **[Planned]**: a `map_path` starting with `./` or `../` resolves against the map file's folder, so worlds need no per-user absolute paths.

3.2 Worlds

[Planned] One hand-edited source file per world, `ros2_ws/maps/src/<world>.yaml`; worlds are named after the room, never after a sensor.

- **Room:** outline polygon, wall height (also the ceiling), wall thickness, wall and ceiling colour.
- ~~**Furniture**~~**Objects**:~~every object is a vertical prism with a height range (`box`, `prism`, and `table` and `chair` macros that expand into prisms), with~~ `box` or `prism`, standing on the floor, with one `height` and a colour. ~~One description gives~~ Every object collides; the LiDAR slice ~~, the collisions and the camera .~~ holds those taller than the laser; the camera draws all of them. One description, so the three never disagree.
- **Track:** a closed centre line from primitives (straights and arcs)~~or a closed point list, or the surveyed tape lines of the real track (`measured_lines`, smoothed),~~ , with the two boundary lines at ±`width`/2; tape width (default 0.05 m) and colour~~; optional seeded gaps where a line is missing~~.
- **Floor:** ~~colour, optional distractor marks, optional photo of the real floor~~one colour.

- ~~**Randomize** (optional, seeded): colour ranges for tape, floor, walls and furniture, applied at start with `appearance_seed`; the geometry never changes.~~
- **Start:** arc length on the centre line and direction; the rear axle is placed there.
- **[Deferred]** (S-15): height ranges (a table seen as legs) and furniture macros, the surveyed tape lines of the real track, missing stretches of tape, colour randomisation, floor marks and a floor photo.

The generator (`ros2 run polimi_sim make_world <source>`) writes, into `ros2_ws/maps/<world>/` (git-ignored):

File	Holds
`room.png`, `.yaml`	Collision map: everything between ground clearance and roof height, the walls, and everything outside the room (black). No tape. Read by the gym.
`room_lidar.png`, `.yaml`	LiDAR slice at laser height: a table is four legs. Read by `lidar_model`.
`scene.yaml`	Exact geometry and colours for the camera, floor included, in `sim_world`.
`tape.yaml`	The tape as polygons and centre polylines: truth for the monitor and the markers.
`centerline.csv`	Centre line with the widths to each side (TUM convention), for practising planning on the simulated track. Named without the map's stem on purpose, so the bridge doesn't count laps itself (S-09).

It also writes the small, tracked `config/maps/<world>.yaml`. Occupancy maps are rasterised conservatively, so ~~a leg~~ an object thinner than a cell stays. Hard checks fail the run: the loop closes (else the gap is printed), ~~the tape is inside the room and clear of furniture, the start is on the centre line, and~~ and every corner is wider than the car's minimum turning radius on that side plus a margin (about 1.02 m left and 1.22 m right with today's steering limits, plus 0.2 m). ~~It prints the lap length and the tape to buy. The launch regenerates a world whose source hash changed~~The launch runs the generator at every start, so a world is never stale.
The shipped world `demo_room` is a placeholder, not the real track: a 10 m×8 m room, a 1.0 m wide loop with a hairpin and a chicane, black tape on a light grey floor, ~~furniture chosen to exercise the models (a box~~ a few boxes (one lower than the ~~camera, a table the LiDAR sees as legs, a chair~~laser, so the LiDAR misses it but the car hits it).

3.3 Model parameters

First guesses from datasheets and upstream defaults. `actuation_model`, `odom_model` and `teleop_bridge` defaults are in `polimi_sim/config/`; the laser's in `config/lidar/`. Live: all `lidar_model` effects, all `actuation_model` and `teleop_bridge` values, the `odom_model` error values. Fixed at start: rates, scan geometry, `range_max`, topics, frames.

3.3.1 `lidar_model` (`config/lidar/sl450.yaml`)

Parameter	Default	Meaning
`rate_hz`	40	SL450 rates: 15, 20, 25, 30, 40 Hz

Parameter	Default	Meaning
`fov_deg`, `resolution_deg`	270, 0.2	1351 beams; `time_increment` $= 1/(\text{rate} \cdot 1800)$, `scan_time` $= 1/\text{rate}$: the sweep lasts 18.75 ms at 40 Hz
`blocks`	8	Groups cast from the truth pose at their own time; 1 = no motion distortion
`stamp_at`, `publish_delay`	`start`, 0.002 s	[VERIFY] against the SL450 driver
`no_return_value`	`'inf'`	REP 117 [VERIFY]
`noise_std`	0.01 m	Range noise
`dropout_prob`	0.002	Random missing beams
`grazing_dropout_prob`, `grazing_max_deg`	0.5, 80°	Loss at grazing incidence
`reflectivity_range_max`	15 m	Low-reflectivity limit
`mixed_pixel_prob`, `mixed_pixel_jump`	0.3, 0.3 m	Mixed pixels at range jumps
`seed`	0	0 = different every run
`lidar_map`	world's `room_lidar`	**[Planned]** The LiDAR slice map, in place of the gym's

Each effect is off at 0.0. Cost: 0.1 ms per scan on the Mac, against a 5 ms budget.

3.3.2 `actuation_model` (`actuation_model.yaml`)

A command does not act at once: it travels to the VESC, the servo needs time to turn, the motor cannot change speed instantly, and the VESC stops the motor when commands stop. The node sits where the VESC would, on a 100 Hz timer, and publishes the steering as sent to the VESC on `/sim/steering_command` (`Float64`, rad).

Parameter	Default	Meaning
`command_delay`	0.02 s	Command transport
`vesc_timeout`	0.5 s	No command for longer: speed 0, steering held [VERIFY] in the VESC app config
`steer_offset`	0.0 rad	Servo trim
`servo_delay`, `servo_rate`	0.03 s, 3.2 rad/s	Servo response
`max_steer`	0.4189 rad	Steering clamp, symmetric today; `max_steer_left` and `max_steer_right` from `car/body.yaml` **[Planned]**
`accel_max`, `brake_max`	4.0, 6.0 m/s^2	Motor limits

All [MEASURE] on the car. The gym still applies its own limits afterwards (steering 3.2 rad/s, acceleration 9.51 m/s^2).

3.3.3 `odom_model` (`odom_model.yaml`)

Mirrors the car's `vesc_to_odom`: `/odom` (frame `odom`, child `base_link`) and TF `odom`→`base_link` at 50 Hz, starting from 0.

- Speed = true longitudinal speed × (1 + `speed_scale_error` 0.02) + noise `speed_noise_std` 0.02 m/s, zeroed below `speed_deadband` 0.05 m/s.
- Yaw rate = $v \tan\delta / L$, with δ the *commanded* steering and `wheelbase` L 0.3302 m (the gym's) [MEASURE]: the car's `vesc.yaml` has 0.25 (`TDD_F1tenth` T-05).
- The yaw rate reads high in corners (9% at 1.5 m/s, steering 0.25 rad): the gym's tyres slip and the formula assumes they do not. `vesc_to_odom` on the car has the same error.
- `debug_map_to_odom`: `off` (default) whenever SLAM or localization runs; `static` fixes `map`→`odom` at the true start pose, so the drift shows (about 8° per 90° corner); `truth` follows the truth like a perfect localizer, so `/scan` stays on the map while `/odom` still drifts. With DEC-10, `truth` also publishes an identity `sim_world`→`map` **[Planned]**.
- `/odom_model/reset` (`std_srvs/Empty`) zeroes `/odom` and re-pins the `static` overlay.

3.3.4 `teleop_bridge` (`teleop_bridge.yaml`)

Keyboard and Foxglove teleop speak `Twist`, the mux speaks Ackermann. `speed = linear.x`; the steering from `angular.z`, as `angular_mode: steering_angle` (rad, default) or `yaw_rate` ($\arctan(L\omega/v)$, $|v|$ never below `min_speed`); clamped to `max_steer`.

3.3.5 `camera_model`

[Planned] The Gemini 335L colour stream as `orbbec_camera` publishes it, with its default camera name `camera`. The frames come from one of three sources, chosen with `camera_source:=`; each publishes the same topics, frames and mount TF, so line detection cannot tell them apart.

Source	Frames	Runs with
`render`	Drawn from the world at the truth pose (below). Default on a map with a `world:` block. Closed loop.	The whole simulator
`video`	A video file or a live stream: `camera_uri:=` is a file path or an RTSP or HTTP URL, read by the same `cv2.VideoCapture` call. A file plays at its recorded frame rate; a stream as frames arrive; stamped on arrival. `camera_info` from `camera_info_url:=`; a frame whose size differs from it stops the node.	The camera source, the mount TF and `foxglove_bridge` only

`bag`	The launch runs `ros2 bag play <camera_uri> -clock` in place of `camera_model`, with the topics, `camera_info` and TF as recorded on the car. If the bag holds only `image_raw/compressed`, the launch adds `image_transport republish` to restore `image_raw`.	As `video`, with `use_sim_time:=true` for every node it starts

- **Replay is open loop:** video and bag frames do not follow the simulated car, so those sources start no gym, models or mux, and only the stack's line detection runs on them, not path following. They test detection on the real tape and the real floor, which the drawing cannot show (S-10).
- **Replay frames:** without `/odom`, `video` gives the path in `base_link` only. A path in `odom` (RASD OI-02) needs a bag that also holds `/tf`, `/tf_static` and `/odom`. The bag keeps the driver's encoding (YUYV on the car), so `polimi_lines` converts through `cv_bridge`.
- **Live camera on the Mac:** Docker Desktop has no USB passthrough, so a webcam or phone streams from the host (for example `ffmpeg` to RTSP) and `video` reads the URL.
- **Order:** `video` and `bag` need neither the gym nor a world, so they are built first, on today's images, ahead of the Jazzy migration (S-05); `render` follows with the room worlds.
- **Drawing:** every visible thing is a flat polygon (floor, tape in chunks of at most 1 m that never overlap, wall panels, every prism face). Per frame: camera pose = truth (rear axle) ∘ `camera_mount.yaml` ∘ the colour optical frame; clip to the view, project with the intrinsics, fill far to near with OpenCV, simple distance shading; ~~the car's nose mask last;~~ lens distortion with one precomputed remap, so the factory calibration (`plumb_bob`~~`or rational_polynomial`~~) can be pasted in.
- **Profiles:** `720p30`~~(default) and `720p60`~~, colour 1280×720 in YUYV on the car; the car's launch pins the same preset. ~~If `720p60` is added once~~ the 335L ~~has no 1280at 60 in YUYV , `720p60` is removed~~ is shown to give it in YUYV (S-06).
- **Output:** `/camera/color/image_raw` (`rgb8`, raw, never compressed for perception) and `camera_info` with the drawing's intrinsics; reliable QoS, keep last 10; stamped at the middle of exposure [VERIFY]; drawn at the truth pose interpolated at the stamp. No noise, blur, latency or dropped frames.
- **Preview:** `/camera/color/image_raw/compressed` (JPEG, 10 Hz, half size), encoded only while someone subscribes, for every source. Never open the raw image in Foxglove: it pushes 83 MB/s through `foxglove_bridge`.
- ~~**Labels** (optional): `/sim/truth/camera/color/label`, a `mono8` class image (floor, tape, wall, furniture, ceiling, nose) for scoring line detectors~~**[Deferred]** (S-14): a class-label image for tuning line segmentation pixel by pixel, the `rational_polynomial` model, the car's nose in the image, colour randomisation. Line detection itself is scored on its path by `track_monitor` (section 3.3.7).
- **Budget:** at most 15 ms per frame at `720p30` on the M4 Pro, on its own thread.
- **Tools:** `render_frame` draws a frame without ROS for a given world and pose; `-footprint` reports the floor area seen and where both lines come into view, to choose the camera mount (S-07).

3.3.6 `imu_model`

~~[Planned]~~**[Deferred]** (S-12). When built: `/camera/gyro_accel/sample` (`sensor_msgs/Imu`), frame `camera_accel_gyro_optical_frame`, 200 Hz, ranges 4 g and 1000°/s with clipping, no orientation, as the driver with synced output. Gyro = yaw rate from the truth twist. Accelerometer = specific force at the IMU: the CoG acceleration (differentiated truth velocity, low-pass) plus the lever-arm terms, plus g along body z, rotated into the optical frame. Noise density, bias random walk and constant bias per axis, seeded [VERIFY]. Physics runs at 100 Hz, so content above 50 Hz is only noise. After a truth jump the differentiator restarts, so contacts give no spikes.

3.3.7 Evaluation

[Planned] From the truth only; the stack never reads it.

- **`track_monitor`:** progress along the centre line, tracked within a window so a hairpin or a chicane cannot jump it to the other part of the track; a lap counts only after all checkpoints (default 10) in order; signed lateral offset ~~; wheels as contact patches from the wheelbase and `track_width`~~of the CoG; off track ~~when all four wheels are out (~~by DEC-14~~); excursions and ~~; collisions per lap; a lap with a collision or a jump is invalid. Topics `/sim/eval/` `on_track`, ~~`wheels_out`,~~ `lateral_offset`, `progress`, `lap`, `lap_time`, `collisions`; one CSV line per lap in `~/ws/runs/` (time, valid, maximum offset, time off track, mean path error).
- **Line detection score** (`track_monitor`, `render` only): each path from line detection goes to `base_link` through the stack's TF at the path's stamp, then to `sim_world` through the truth pose at the same stamp, so odometry drift cancels and the score holds whether OI-02 picks `base_link` or `odom`. The distance of its points to the tape centre line gives `/sim/eval/path_error_mean` and `path_error_max` (m). The path must carry the image's stamp, or latency enters the score. Replay has no truth: there, detection is judged by eye in Foxglove.

- **Simulator health:** `/sim/health/physics_rate` (distinct truth poses per second while moving, expected 100) and `odom_rate` (expected 250), with a warning below 95 %, so rendering that starves the gym shows.
- **`localization_monitor`~~:~~ ~~[Deferred]~~** (S-13), with option B: the stack's `map`→`base_link` against the truth ~~, matched by stamp,~~ after a fitted SE(2) alignment~~(no scale) published as TF so Foxglove overlays both; `/sim/eval/loc/` position and heading error;~~ , errors per lap and per speed bin~~(1, 2, 3, 4 m/s) mean, p95 and max in a CSV. It scores~~ . It would score AC-V2-B in the simulator.
- `ros2_ws/runs/`, git-ignored, is bind-mounted at `~/ws/runs` in every image.

3.4 Ideal mode

`ideal:=true` keeps the nodes, topics and frames and switches the imperfections off (DEC-06).

Node	Overrides
`lidar_model`	`effects_enabled: false`, `blocks: 1`, `publish_delay: 0.0`
`actuation_model`	`command_delay`, `servo_delay`, `steer_offset`: 0.0; `servo_rate`, `accel_max`, `brake_max`: 1000.0 (the gym's own limits remain)
`odom_model`	`speed_scale_error`, `speed_noise_std`, `speed_deadband`: 0.0
`debug_map_to_odom`	`truth`, unless set on the command line

Still there in ideal mode: the 40 Hz scan with rotating-sensor time fields and `inf` beyond 25 m; the mux timeout (0.2 s), the VESC timeout (0.5 s) and the steering limits; the `/odom` yaw drift from tyre slip. The camera has no imperfections in either mode.

4 Interfaces

4.1 Simulator versus car

	Gym alone	`simcar`	Car
World	a track bounded by walls	today the same; **[Planned]** a room with furniture and a taped loop on the floor	the room
`/scan`	250 Hz, all beams at one instant, frame `ego_racecar/laser`	40 Hz, rotating-sensor timing and motion distortion, frame `laser`; **[Planned]** the room's slice at laser height, tape invisible	SL450 driver
Scan errors	Gaussian noise	noise, dropouts, grazing loss, 15 m low-reflectivity limit, mixed pixels	real
Camera	none	**[Planned]** the 335L's colour stream, topics and geometry, drawn without imperfections; or a replayed video, stream or bag, open loop	Gemini 335L
IMU	none	**[Deferred]** planar motion, lever arm, noise and bias (S-12)	Gemini 335L IMU
Command path	`/drive` straight into physics, held forever	`/drive` → `ackermann_mux` → `actuation_model`	mux → VESC
Actuation	ideal	command delay, VESC timeout, servo delay and rate, accel and brake limits	VESC, servo
Odometry	truth in `map`	speed scale error and noise, yaw rate from the commanded steering; drifts	`vesc_to_odom`
Frames	`map`→ `ego_racecar/base_link`→ `ego_racecar/laser`	`odom`→`base_link`→`laser`, truth tree apart (under `sim_world`, **[Planned]**)	`odom`→ `base_link`→ `laser`
Teleop	`/cmd_vel` into physics	`/cmd_vel` → `teleop_bridge` → `/teleop` → mux	joystick → `/teleop` → mux
Leaving the track	a wall collision	**[Planned]** nothing physical; scored by `track_monitor`	nothing physical; judged by people

4.2 Topics

The stack may use `/scan`, `/odom`, the camera and IMU topics, TF `odom`→`base_link` and the sensor frames, and publish `/drive`. It never reads `ego_racecar/*`, `/ego_racecar/odom`, `/sim/*` or the `sim_world` frame: that is the ground truth, which the car does not have (FR-SIM-003), and it never publishes on `/sim/*`.

Topic	Type	Direction
`/scan`	`LaserScan`, frame `laser`, 40 Hz	out, `lidar_model`
`/odom`	`Odometry`, frame `odom`, 50 Hz	out, `odom_model`
`/camera/color/image_raw`, `camera_info`	**[Planned]** `Image` `rgb8`, `CameraInfo`; 30 Hz	out, `camera_model` or the bag
`/camera/color/image_raw/compressed`	**[Planned]** `CompressedImage`, preview	out, only while subscribed
`/drive`	`AckermannDriveStamped`	in, autonomy, mux priority 10
`/teleop`	`AckermannDriveStamped`	in, mux priority 100, from `teleop_bridge`
`/cmd_vel`	`Twist`	in, teleop
`/initialpose`	`PoseWithCovarianceStamped`	in, today the reset of the true pose; **[Planned]** AMCL's, as on the car
`/map`	`OccupancyGrid`	out today; **[Planned]** SLAM's and AMCL's, as on the car
`/sim/map`, `/sim/initialpose`	**[Planned]** the gym's map and reset input	truth and simulator control
`/sim/reset`, `/sim/reset_pose`	**[Planned]** `Empty`, `PoseStamped` (rear axle)	in, `sim_reset`, for people and scripts
`/sim/truth/tape`	**[Planned]** `MarkerArray`	out, evaluation only
`/sim/eval/*`, `/sim/health/*`	**[Planned]** `std_msgs` values	out, evaluation only
`/ego_racecar/odom`	`Odometry`, truth	out, evaluation only
`/ego_racecar/collision`	`Bool`	out, evaluation only
`/ackermann_cmd`, `/sim/drive`, `/sim/steering_command`, `/sim/scan_unused`	internal	–

Every pose the simulator takes or gives (`sx`, `sy`, `stheta`, `/sim/initialpose`, `/sim/reset_pose`, the sensor mounts, the start line) is the rear axle, `base_link` on the car, once on the `dev-jazzy` bridge.

5 Requirements Traceability

Requirement	Component(s)	Test(s)
`FR-SIM-001`	model nodes, camera model, configs (DEC-01, DEC-02, DEC-11); IMU model deferred (S-12)	`ros2 topic info` and `tf2_echo` against the car interface
`FR-SIM-002`	mux, `actuation_model` timeouts; locks and AEB with `TDD_F1tenth`	AC-V1-04 in the simulator

FR-SIM-003	two frame trees, `/sim` namespace (DEC-04, DEC-10)	no stack node subscribes to `ego_racecar/*`, `/sim/*` or looks up `sim_world`
NFR-POR-001	`arm` and `x86` images	the same `simcar` on both
FR-LIN-001	`camera_model` (all sources), `track_monitor`	path error with `render`; recordings of the real tape with `video` and `bag`
AC-V1-02	room world, `camera_model`, `track_monitor`	laps on track in the simulator, measured by `/sim/eval/*`
AC-V2-B	`localization_monitor`, deferred with option B (S-13)	fused pose against the truth per speed bin

6 Testing

Level	Scope	Tooling / gate
Unit	Model logic, plain Python without ROS, on synthetic data: models, world generator, camera geometry, monitors	pytest in `polimi_sim/test`; reference images from `render_frame`
Comparison	Each model against the gym alone	ideal mode against the normal run; `effects_enabled false` against `/sim/scan_unused` (they match standing still except where the gym rounds beam angles to a table)
Detection	Line detection on each camera source	`/sim/eval/path_error_*` per lap with `render`; by eye in Foxglove on replayed recordings of the real tape
Health	The simulator keeps real time	`/sim/health/*` $\geq$ 95 % with the camera on
System	The stack on the simulator	AC-V1-02 and 04 in the simulator first (`RASD_F1tenth`)

7 Known Limits

Where the simulator still differs from the car, or simplifies. Today:

- No camera, no IMU, and no room with a taped track: the maps are tracks bounded by walls, which the project's environment no longer has (`RASD_F1tenth` C-001 to C-005).
- The gym's map and reset input share `/map` and `/initialpose` with SLAM and AMCL, and the truth hangs from `map`: giving AMCL an initial pose would teleport the car. Fixed by DEC-10.
- Truth pose: the gym's single-track state is at the centre of gravity, published as `ego_racecar/base_link`. `polimi_sim` treats it as `base_link` (the rear axle on the car): a 0.17 m offset, gone on the `dev-jazzy` bridge.
- Steering is clamped at $\pm$0.4189 rad; the car's servo clamp gives +0.31 rad left and $-$0.26 rad right, so the simulated car turns tighter than the real one (`TDD_F1tenth` T-12; `car/body.yaml` **[Planned]**).
- Vehicle body parameters are the gym's `f1tenth` preset.

- The truth pose changes at 100 Hz (the physics step). `lidar_model` interpolates between steps and extrapolates up to 30 ms past the newest, and today across resets too.
- Grazing loss and mixed pixels come from neighbouring beams, not from the map geometry.
- `lidar_model` needs the bridge on wall-clock time: `use_sim_time: False` in `sim/gym.yaml`.
- The gym's 3D car model and wheels stay on the `ego_racecar/*` frames.
- A wall hit makes the truth jump: the gym zeroes the car's heading on collision; `/odom` does not follow. The JAX gym pushes the car out instead.
- `/initialpose` alone moves the truth, not `/odom`: wheel odometry keeps integrating, as on the car.

By design, also after the planned work:

- Flat floor; objects are vertical prisms standing on the floor (no slopes, no curved furniture, nothing to drive under).
- The camera draws flat-shaded polygons: no noise, blur, latency or dropped frames, no lighting beyond simple shading, no shadows or glare, no auto-exposure; the floor is one colour unless a photo is used; large overlapping objects can be drawn in the wrong order. No IR, depth or point cloud.
- Replayed camera sources are open loop: the frames do not follow the simulated car, so they test detection only, and they have no truth to score against.
- No body pitch or roll: the camera pose is the static mount, ~~the IMU is planar,~~ and the LiDAR stays a planar slice at the nominal laser height.
- ~~Camera and IMU~~ No IMU until S-12. Camera parameters are nominal until calibrated.

8 Open Items

ID	Topic	Notes
S-01	Deadman and e-stop stand-in	No USB passthrough on Docker Desktop for macOS, so no joystick in the container. A `/joy` stand-in is built with the safety locks (`TDD_F1tenth` T-01).
S-02	Measured values [MEASURE]	Laser and camera mounts, actuation parameters, wheelbase, steering limits, track width, tyre patch, roof height, from the car once built.
S-03	Driver behaviour [VERIFY]	SL450: `stamp_at`, `publish_delay`, `no_return_value`; the VESC timeout; Gemini 335L: distortion model in `camera_info`, image stamp point, IMU orientation field.
S-04	Real room and track [DECIDE]	Layout, tape, floor and room; `demo_room` stands in until surveyed.
S-05	Migration	The Jazzy migration in `TDD_F1tenth` comes first: the drawn camera, IMU and room worlds are built on the JAX gym and the fork. The camera's `video` and `bag` sources use no gym and come first. The mux leaves with DEC-09.
S-06	Camera profile [VERIFY]	Whether the 335L gives 1280×720 at 60 fps in YUYV; if not, `720p60` is removed and everything stays on `720p30`.
S-07	Camera mount [DECIDE]	Height, pitch and position, chosen with `render_frame -footprint` before the bracket is designed, then measured.
S-09	Laps [DECIDE]	Keep `track_monitor`'s laps (ordered checkpoints, valid flag) or use the `dev-jazzy` bridge's `lap_count` and `lap_time`.

ID	Topic	Notes
S-10	Camera imperfections (optional)	Deferred: noise, blur, YUYV colour, latency, drops, body pitch, IR and depth are added only when a real-versus-simulated comparison shows the stack needs one.
S-11	Contacts on the JAX gym [VERIFY]	How `/ego_racecar/collision` reports a contact (one edge per contact or every step), for the truth-jump detector.
S-12	IMU model **[Deferred]**	Build `imu_model` (section 3.3.6) when RASD OI-08 is decided yes (EKF in the base platform) or option B is chosen.
S-13	Localization monitor **[Deferred]**	Build with option B, to score AC-V2-B in the simulator.
S-14	Camera extras **[Deferred]**	Label images (for tuning segmentation; detection is scored on the path), `rational_polynomial`, the car's nose in the image, colour randomisation: each when a consumer exists.
S-15	World extras **[Deferred]**	Height ranges and furniture macros, the surveyed tape lines (with S-04), missing stretches of tape, floor marks and photo.